Binary classification - Logistic regression

Phuong Luu Vo

September 15, 2026

1 Sigmoid function

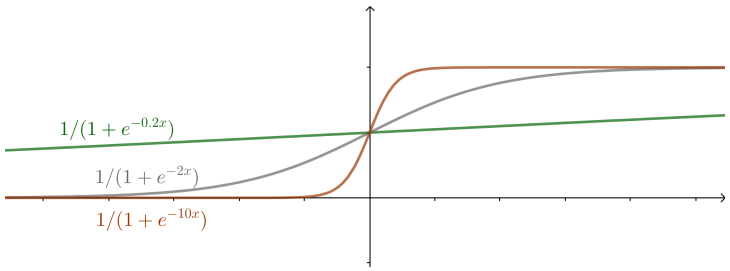


Figure 1: Sigmoid function $\frac{1}{1+e^{-ax}}$ with $a = 0.2, 2, 10$.

Sigmoid function is defined as follow:

$$\sigma(a) = \frac{1}{1+e^{-a}}.$$

Some properties of sigmoid function:

- $0 < \sigma(a) < 1$, hence, we apply sigmoid function to model a probability.
- $\sigma(-a) = 1 - \sigma(a)$.
- Inverse function is called *logit*,

$$a = \log(\frac{\sigma}{1-\sigma})$$

- Derivative

$$\frac{d\sigma}{da} = \sigma(1-\sigma).$$

2 Logistic regression

Given dataset (X, y), in which $X = \begin{bmatrix} -x_1^T- \\ ... \\ -x_N^T- \end{bmatrix}$ is the matrix with N observations, $y = \begin{bmatrix} y_1^T \\ ... \\ y_N \end{bmatrix}$, where y_i is 0 or 1.

We use sigmoid function with parameter w to model the probability of an observation (x_i, y_i):

$$P(y_i = 1|x_i) = \pi_i = \sigma(w^T x_i). \tag{1}$$

Hence,

$$P(y_i = 0|x) = 1 - \pi_i. \tag{2}$$

We rewrite the likelihood for observation x_i, y_i as follows:

$$P(y_i|x_i) = \pi_i^{y_i}(1 - \pi_i)^{1-y_i}. \tag{3}$$

Hence, the likelihood is:

$$P(y|X) = \prod_{i=1}^{N} \sigma(w^T x_i)^{y_i}(1 - \sigma(w^T x_i))^{1-y_i}. \tag{4}$$

We need to find w to maximize likelihood $P(y|X; w)$ (maximum likelihood estimation).

Logarithm transforms a product to a summation. The **log-likelihood function** is given by

$$\begin{aligned} \log(P(y|X)) &= \sum_{i=1}^{N} y_i \log(\pi_i) + (1 - y_i) \log(1 - \pi_i) = -\sum_{i=1}^{N} BCE(y_i, \pi_i) \\ &= \sum_{i=1}^{N} y_i w^T x_i + \log(1 - \sigma(w^T x_i)) \\ &= \sum_{i=1}^{N} y_i w^T x_i - \log(1 + e^{w^T x_i}). \end{aligned}$$

where $BCE(\dots, \dots)$ is the cross-entropy between two Bernoulli distributions.

Hence, the optimization problem of logistic regression is:

$$\max. \sum_{i=1}^{N} y_i w^T x_i - \log(1 + e^{w^T x_i}) \tag{5}$$

$$\text{or} \quad \min. \sum_{i=1}^{N} BCE(\boldsymbol{y}_i, \boldsymbol{\pi}_i). \tag{6}$$

3 Example of logistic regression in CVXPY

The example with artificial dataset.

```
import cvxpy as cp
import numpy as np
import matplotlib.pyplot as plt
```

Create 100 data points for training and 100 others for testing. x_i has 02 features, y_i is 0 or 1.

```
np.random.seed(1)
d = 2     # number of features
m = 100   # number of samples

weight_true = (np.random.random(d) - 0.5)*2
X = (np.random.random((m, d)) - 0.5)*5
Y = np.round(sigmoid(X @ weight_true + np.random.randn(m)*0.5))

m_test=m
X_test = (np.random.random((m_test, d)) - 0.5)*5
Y_test = np.round(sigmoid(X_test @ weight_true + np.random.randn(m_test)*0.5))
```

Verify the shape of X and Y:

```
print(X.shape, Y.shape)
```

```
(100, 2) (100,)
```

We can plot the data points on 2D:

```
plt.scatter(X[Y == 0, 0], X[Y == 0, 1], marker='o')
plt.scatter(X[Y == 1, 0], X[Y == 1, 1], marker='v')
plt.xlabel("$x_1$", fontsize=16)
plt.ylabel("$x_2$", fontsize=16)
plt.savefig("train_dataset.png")
```

Input the problem to CVX

```
weight = cp.Variable(d)
weight0 = cp.Variable()
log_likelihood = cp.sum(
    cp.multiply(Y, X @ weight + weight0) - cp.logistic(X @ weight + weight0)
)

problem = cp.Problem(cp.Maximize(log_likelihood))
problem.solve()

print(weight0.value)
print(weight.value)
```

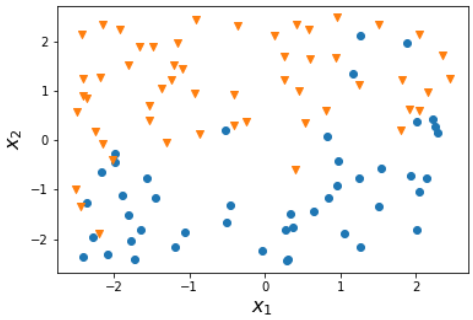


Figure 2: Dataset in 2D.

The optimal weight is:

```
0.03178046482109968
[−0.73230372   1.93465795]
```

Calculate error:

```
def error(scores, labels):
def error(scores, labels):
  scores[scores >= 0.5] = 1
  scores[scores < 0.5] = 0
  return scores, np.sum(np.abs(scores - labels)) / float(np.size(labels))

_, error_train = error( sigmoid(X @ weight.value + weight0.value), Y)
Y_predict, error_test = error( sigmoid(X_test @ weight.value + weight0.value), Y
print("Train error:", error_train)
print ("Test error:", error_test)
```

Results:

```
Train error:  0.14
Test error:  0.17
```

Plot in 2D

```
plt.scatter(X_test[Y_test == 0, 0], X_test[Y_test == 0, 1], marker='o')
plt.scatter(X_test[Y_test == 1, 0], X_test[Y_test == 1, 1], marker='v')
plt.xlabel("$x_1$", fontsize=16)
plt.ylabel("$x_2$", fontsize=16)
plt.savefig("test_dataset.png")

plt.scatter(X_test[Y_predict == 0, 0], X_test[Y_predict == 0, 1], marker='o')
plt.scatter(X_test[Y_predict == 1, 0], X_test[Y_predict == 1, 1], marker='v')
plt.xlabel("$x_1$", fontsize=16)
plt.ylabel("$x_2$", fontsize=16)
plt.savefig("prediction.png")
```

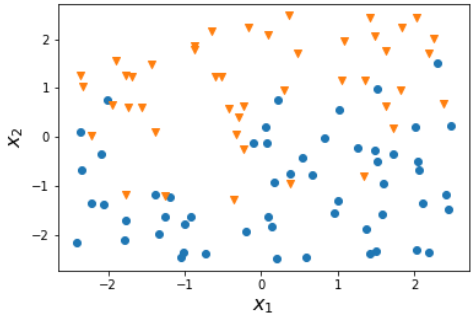


Figure 3: Test set.

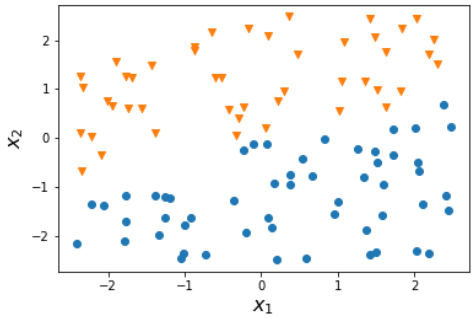


Figure 4: Results of classification by logistic regression model.

4 Logistic regression with ℓ_1 norm regularization

Understand and run the code in this page: `https://www.cvxpy.org/examples/machine_learning/logistic_regression.html`.

References

[1] Stephen P. Boyd, and Lieven Vandenberghe. Convex optimization. Cambridge university press, 2004.

[2] Logistic regression with ℓ_1 norm regularization. `https://www.cvxpy.org/examples/machine_learning/logistic_regression.html`.